

MySQL + Redis 面试高频 25 题

(AI 播客文字稿 / 睡前复习版)

生成日期：2026-02-08 | 适配：口播 30-45 分钟

使用方式：把这份稿子交给你的 TTS / AI 播客工具直接朗读。建议按 “问题 -> 答案 -> 追问” 节奏口播，遇到关键词（如 MVCC、next-key lock）停 1 秒。

与你简历强相关的重点（面试一定会追问）

- | |
|--|
| • 你在简历 “专业技能” 中明确写了：掌握 MySQL（索引）与 Redis（分布式锁/缓存）。 |
| • 你在 “智能体平台/记忆中枢” 经历里写到：通过 Redis + Lua 分布式锁 解决高并发竞争。 |
| • 你在 “分布式即时通讯项目” 里写到：封装 MySQL/Redis 连接池，采用 Cache-Aside 与 延时双删 保障缓存一致性。 |

接下来进入 25 个高频题：MySQL 13 题 + Redis 12 题。

目录

模块	覆盖要点
MySQL（13题）	索引/索引失效/覆盖索引/ICP/ACID/MVCC/隔离级别/锁/日志/两阶段提交/更新流程
Redis（12题）	数据类型/为什么快/线程模型/IO多路复用/跳表/SDS/持久化/过期与淘汰/缓存三大问

注：目录为口播提示，不是死背。面试时把“ 机制-现象-取舍 ”讲出来，效果最好。

MySQL 高频题

Q1：MySQL 里索引是什么？为什么能加速查询？

- 索引是帮助 MySQL 高效获取数据的数据结构，本质上是“有序结构 + 指针/页引用”。InnoDB 常见的是 B+Tree。
- 把“全表扫描 $O(N)$ ”变成“树查找 $O(\log N)$ ”；同时利用索引有序性加速排序/分组/范围查询。

Q2：InnoDB 的聚簇索引 vs 二级索引有什么区别？回表是什么？

- 聚簇索引（主键索引）叶子节点存的是整行数据；二级索引叶子节点存的是“二级索引键 + 主键值”。
- 用二级索引查到主键后，再去聚簇索引取整行，就是回表。优化方向：覆盖索引、减少回表、I CP（索引下推）。

Q3：覆盖索引是什么？什么时候特别值钱？

- 查询列都能从二级索引叶子节点拿到，不需要回表。
- 典型：只查 id 或少量字段的列表页；或配合联合索引让查询“只走索引”。

Q4：联合索引为什么要遵守最左前缀？字段顺序怎么选？

- 联合索引按 (a,b,c) 排序，能连续利用前缀：a、(a,b)、(a,b,c)。跳过 b 直接用 c，会导致 c 不能用于定位。
- 顺序建议：把区分度高、过滤性强、最常用的条件放前面；范围条件一般放后面。

Q5：索引下推（ICP）解决了什么？

- 当联合索引只能用到前缀（比如 $a=2$ and $c=1$ ），传统方案需要回表后再过滤 c；ICP 会把 c 的过滤下推到“索引层”先过滤，减少回表。

Q6：常见索引失效有哪些？怎么快速自查？

- 高频 6 类：like 左模糊/左右模糊（ $\%xx/\%xx\%$ ）、对索引列使用函数、对索引列做表达式计算、隐式类型转换、联合索引不满足最左前缀、优化器认为走索引收益不高（如区分度太低）。
- 自查路径：EXPLAIN 看 type/key/rows；慢查询日志定位；必要时加上正确的联合索引与覆盖索引。

Q7：事务的 ACID 分别在 InnoDB 里靠什么实现？

- A 原子性：undo log + 锁，失败可回滚；C 一致性：由 A/I/D 共同保证；I 隔离性：MVCC + 锁；D 持久性：redo log + WAL（先写日志再落盘）。
- 面试点：不要只背概念，要能说出“对应机制”。

Q8：MySQL 默认隔离级别是什么？可重复读靠什么实现？

- 默认是 Repeatable Read（可重复读）。
- 实现依赖 MVCC：事务第一次一致性读时生成 Read View，后续用 Read View 沿 undo log 版本链读到“事务开始时的快照”。

Q9：可重复读为什么还可能出现幻读？怎么处理？

- 一致性读（普通 SELECT）基本避免幻读，但“当前读/锁定读”（UPDATE/SELECT ... FOR UPDATE）会加锁并可能让之前不可见的记录变得可见，从而出现典型幻读现象。
- 处理：需要严格避免时，用 SELECT ... FOR UPDATE 等锁定读，配合 next-key lock 防插入。

Q10：next-key lock 是什么？它在解决什么问题？

- next-key lock = 记录锁 + 间隙锁；锁住一段范围，防止别的事务在该范围插入新记录。
- 核心目的：在 RR 下避免“范围条件”的幻读与并发插入。

Q11：一条 UPDATE 的执行流程是怎样的？（从 buffer pool 到日志）

- 若数据页不在 buffer pool，先读入；执行器判断是否真的变化；InnoDB 先记录 undo log（可回滚、MVCC 用）；再更新内存并写 redo log（WAL）。
- 脏页异步刷盘；事务提交时再配合日志持久化策略保证 D。

Q12：redo log / undo log / binlog 各自干啥？两阶段提交解决什么？

- redo：崩溃恢复（D）；undo：回滚与 MVCC（A + I）；binlog：Server 层逻辑日志，用于备份与主从复制。
- 两阶段提交：保证 redo 与 binlog 一致（prepare 写 redo + XID；commit 写 binlog 再把 redo 标 commit；恢复时按 XID 判定提交/回滚）。

Q13：为什么“update 没走索引”会更容易把系统锁死？

- 如果 update 触发全表扫描，会在扫描过程中对大量记录加锁（记录锁+间隙锁），等价于“锁住全表”，其他更新/锁定读容易被阻塞。
- 面试回答要点：不仅是慢，更是并发被锁放大。

Redis 高频题

Q14：Redis 支持哪些数据类型？各自适合什么场景？

- 常见 5 种：String/Hash/List/Set/Zset；扩展：BitMap/HyperLogLog/GEO/Stream。
- 把“数据结构 + 场景”绑定记：String 计数/锁，Hash 对象缓存，List 队列，Set 去重与集合运算，Zset 排行榜。

Q15：Redis 为什么快？把答案讲成 3 个点就够了

- 内存读写（消除磁盘 IO）；高效数据结构（哈希 O(1) 查找等）；单线程事件循环 + IO 多路复用减少上下文切换与锁竞争。

Q16：Redis 真的是单线程吗？哪些地方用到了多线程？

- 命令执行主流程长期是单线程；但有后台线程（BIO）处理 AOF 刷盘、释放内存等；Redis 6.0+ 引入 IO 多线程处理网络读写以提升吞吐。

Q17：IO 多路复用在 Redis 里怎么串起来的？

- 客户端连接对应 FD
注册到多路复用器（select/epoll），就绪后回调文件事件处理器，再进入命令执行。
- 把它讲成 Reactor：事件驱动 + 单线程轮询。

Q18：Zset 为啥用跳表，而不是 B+Tree？

- 二者查找都是 $O(\log N)$ ，但跳表实现简单、写入局部修改、适合内存指针跳转；B+Tree 更偏“磁盘页”优化，维护复杂，写入可能触发分裂/合并带来抖动。

Q19：SDS 是什么？它比 C 字符串强在哪？

- SDS 是 Redis 的字符串实现，额外保存 len/alloc 等元信息。
- 优势： $O(1)$ 取长度、二进制安全、不易缓冲区溢出（自动扩容）。

Q20：AOF 和 RDB 怎么选？常用写回策略怎么说？

- RDB：定时快照，体积小恢复快；AOF：记录写命令，数据更安全但文件更大。
- AOF 写回策略：Always（最安全最慢）、Everysec（常用折中，最多丢 1s）、No（交给 OS，性能好但风险大）。

Q21：Redis 的过期删除策略 vs 内存淘汰策略，有什么区别？

- 过期删除：键到了 TTL 怎么删（惰性删除 + 定期删除）。
- 内存淘汰：内存满了删谁（如 allkeys-lru / volatile-lru 等策略）。

Q22：缓存穿透、击穿、雪崩分别是什么？一套“面试顺口溜”

- 穿透：查不存在 -> 都打到 DB（解：布隆过滤器/空值缓存/参数校验）。
- 击穿：热点 key 失效瞬间大量并发打 DB（解：互斥锁/单飞、逻辑过期）。
- 雪崩：大量 key 同时过期或 Redis 故障（解：过期时间加随机、分级缓存、限流降级、主从/哨兵/集群）。

Q23：布隆过滤器怎么讲？它的误判特点是什么？

- 用多个哈希把元素映射到位图并置 1；查询时对应位都为 1 才“可能存在”。
- 特点：可能误判存在，但不会误判不存在；适合挡住“必不存在”的穿透流量。

Q24：分布式锁怎么用 Redis 做？为什么经常配 Lua？

- 核心：SET key value NX PX ttl 作为加锁；value 用随机串标识持有者；解锁必须校验 value 再 DEL。
- 用 Lua：把“校验 + 删除”做成原子操作，避免误删别人的锁。进一步可做续期/看门狗。

Q25：简历项目里 Cache-Aside + 延时双删 怎么讲才能更像实战？

- 读：先查 Redis，miss 再查 MySQL 并回填缓存；写：先写 DB，再删缓存；延时双删：写 DB 后删一次，再延迟一段时间再删一次，降低并发下脏缓存概率。
- 补充口径：延时窗口要结合业务写放大/并发；并配合消息队列或 binlog 订阅做最终一致性。

睡前 5 分钟复习法

- 把 25 题分成 3 轮：第一轮只听问题，能说出关键词就过；第二轮听答案，补全逻辑链；第三轮挑 5 题做“追问扩展”。
- 重点背“句式模板”：比如 MySQL 事务 -> 先说 ACID，再说对应机制（undo/redo/MVCC/锁）。
- 面试官追问时，优先用你简历里的场景落地：连接池、Cache-Aside、延时双删、Redis+Lua 分布式锁。

祝你今晚复习顺利，明天面试一口气把“原理+实战”讲透。